NAME : ARLEEN JESSICA A
REG NO : 192519020
COURSE CODE : CSA0820
COURSE NAME : PYTHON PROGRAMMING

ASSIGNMENT_CO 1

TASK 1

Aim :
To analyse the milk payment calculation problem and identify the required input values, output values, variables, constants and data types. The task also aims to understand how Python can be used to represent real-world information and organise it for further processing.

Problem Statement :
Milk collection centres receive milk from farmers every day. The payment made to each farmer depends on the quantity of milk supplied, the fat percentage and the Solids-Not-Fat (SNF) percentage. Calculating the payment manually for many farmers can be time-consuming and may lead to errors.
The objective of this task is to study the requirements of a milk payment calculator before developing the complete program. The program must collect the farmer's identification number, quantity of milk, fat percentage and SNF percentage. These values will later be used to calculate the rate per litre and the total payment.
The standard fat percentage is taken as 3.5%, the standard SNF percentage as 8.5%, and the base rate as ₹30 per litre. Each 0.1% change in fat changes the rate by ₹0.40, while each 0.1% change in SNF changes the rate by ₹0.30.The program must also identify invalid values, such as negative milk quantity or a fat percentage outside the permitted range of 2.0% to 7.0%.

Data Types :
Python provides different data types to store different kinds of information. The following data types are relevant to the milk payment calculator.

Data Type	**Purpose**	**Example**
str	Stores farmer ID	“F101”
int	Stores whole numbers	10
float	Stores decimal measurements	3.5
bool	Stores logical results	True
list	Stores multiple records	[10, 3.5, 8.5]

The farmer ID is stored as a string because it may contain letters and numbers. Milk quantity, fat percentage and SNF percentage are stored as floating-point values because they may contain decimal places.Constants are values that are intended to remain unchanged during program execution. Python conventionally represents constants using uppercase names.

Input :

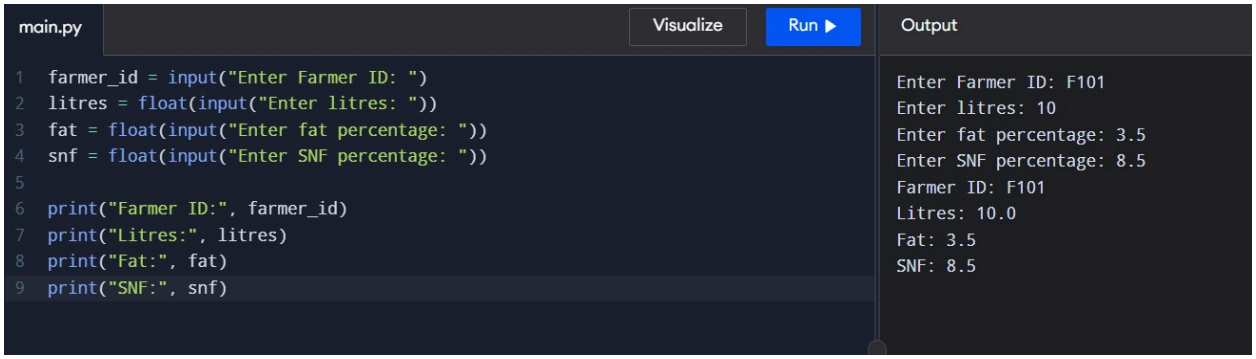
The program accepts the following input values:

- Farmer ID
- Quantity of milk in litres
- Fat percentage
- SNF percentage

The input() function is used to collect information from the user. Numerical input must be converted into the appropriate numerical type before calculations are performed.

Output :

The expected output consists of the farmer ID, quantity of milk, fat percentage and SNF percentage entered by the user. These details provide the basic information required for later payment calculations.



Evaluation :

The program is evaluated by checking whether it accepts and displays the input values correctly. The farmer ID must remain a string, while numerical values must be converted correctly.

For example, if the farmer supplies 10 litres of milk with 3.5% fat and 8.5% SNF, the program should store these values without changing their meaning.

The following Python program demonstrates the collection and display of the input values.

Conclusion :

The milk payment calculation problem was analysed successfully. The required inputs, outputs, variables, constants and data types were identified. This task establishes the foundation for developing a complete milk payment calculator using Python. Proper identification of data before programming improves the accuracy, readability and maintainability of the final application.

TASK 2

Aim :
To study Python data types, arithmetic operators, numerical expressions and floating-point calculations. This task also aims to compare floating-point arithmetic with decimal arithmetic and understand the importance of numerical accuracy in financial applications.

Problem Statement :
Python supports different types of values, including integers, floating-point numbers, strings and Boolean values. These types are used for storing and processing information in a program.
In a milk payment calculator, numerical calculations are required to determine the rate per litre and the total payment. However, ordinary floating-point calculations may produce small representation errors because some decimal fractions cannot be represented exactly in binary floating-point format.
The objective of this task is to demonstrate basic Python data types and arithmetic operations, and to compare the results of floating-point arithmetic with those obtained using the Decimal class.

Data Types :
The main data types used in this task are:
Integer (int): Represents whole numbers without decimal points. Example: 10.
Floating-point (float): Represents numbers with decimal points. Example: 10.5.
String (str): Represents text. Example: "Farmer101".
Boolean (bool): Represents logical values, either True or False.
Decimal (Decimal): Represents decimal values using decimal arithmetic. It is useful for calculations involving money when initialized appropriately.

Input
This task uses predefined values directly in the program. The values include an integer, a floating-point number, a string and a Boolean value. The decimal arithmetic example also uses decimal values represented as strings.

Output :
The program displays the data type of each value. A separate program demonstrates the results of arithmetic operations and decimal addition.

main.py

Visualize Run

```
print(type(10))
print(type(10.5))
print(type("Hello"))
print(type(True))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

main.py

Visualize Run

```
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(7 ** 2)
```

Output

```
3.5
3
1
49
```

main.py

Visualize Run

```
from decimal import Decimal

print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)

a = Decimal("0.1")
b = Decimal("0.2")

print(a + b)
print(a + b == Decimal("0.3"))
```

Output

```
0.30000000000000004
False
0.3
True
```

Evaluation :

The output shows that Python correctly identifies integers, floating-point numbers, strings and Boolean values. Arithmetic operators also produce the expected results for the selected examples.

The floating-point addition of 0.1 and 0.2 produces a value slightly different from 0.3. This happens because these decimal fractions cannot be represented exactly in binary floating-point format.

When Decimal is constructed from strings, the addition produces exactly 0.3. This makes decimal arithmetic suitable for applications where decimal accuracy is important, such as calculating payments.

The comparison also demonstrates why numerical values should be selected carefully according to the requirements of an application. Floating-point arithmetic is suitable for many scientific calculations, while decimal arithmetic is often preferable for financial values.

Conclusion :

Different Python data types and arithmetic operators were studied successfully. The comparison between floating-point and decimal arithmetic demonstrated the importance of numerical precision. The task also established that choosing the correct data type helps improve the reliability of calculations in the milk payment application.

TASK 3

Aim :
To understand variable declaration, constant naming conventions, multiple assignment and dynamic typing in Python, and to apply these concepts to the milk payment calculator.

Problem Statement :
A Python program requires variables to store input values and intermediate results. In the milk payment calculator, values such as farmer ID, milk quantity, fat percentage and SNF percentage must be stored before calculations can be performed.
The program also uses fixed values, such as the base rate and standard fat and SNF percentages. If these values are written repeatedly throughout the program, modifying the calculation rules may become difficult.
The objective of this task is to create meaningful variable names and named constants. It also demonstrates how multiple assignments and dynamic typing work in Python.

Data Types :
The following data types are used:

Variable	**Data type**	**Purpose**
farmer_id	int	Stores the farmer identification number
litres	Int or float	Stores the quantity of milk
fat	float	Stores fat percentage
snf	float	Stores SNF percentage
BASE_RATE	float	Stores the base rate
STANDARD_FAT	float	Stores the standard fat percentage
STANDARD_SNF	float	Stores the standard SNF percentage

A variable is a named reference to a value. A constant is a value intended to remain unchanged throughout the program. Python does not strictly enforce constants, but uppercase naming is the usual convention.

Input :
The program uses predefined values to demonstrate variable assignment. The farmer ID is assigned a string, while the milk quantity, fat percentage and SNF percentage are assigned numerical values.

Output :
The output displays the farmer ID, base rate, milk quantity, fat percentage and SNF percentage.



Evaluation :
Python uses dynamic typing, which means that a variable can refer to values of different data types during program execution. For example, a variable may initially refer to an integer and later refer to a string.
Named constants improve maintainability. If the base rate changes, modifying BASE_RATE in one location is easier than searching for repeated numerical values throughout the program.

Conclusion :
Variables and constants were created and used successfully. Different data types, multiple assignments and dynamic typing were demonstrated. The use of meaningful variable names and named constants improves the readability, consistency and maintainability of the milk payment calculator.

TASK 4

Aim :
To construct and evaluate arithmetic expressions using Python operators to calculate fat steps, SNF steps, rate per litre and the final payment amount.

Problem Statement :
The milk payment calculator must determine the rate per litre based on the difference between the measured fat and SNF percentages and their standard values.
Arithmetic expressions are used to calculate the number of fat and SNF steps. These step values are then multiplied by their corresponding rate adjustments and added to the base rate.
The objective of this task is to construct correct arithmetic expressions and understand how operator precedence and rounding affect the final result.

Data Types :
- float — Stores fat percentage, SNF percentage, base rate and payment.
- int — Stores rounded fat steps and SNF steps.
- bool — Represents the result of comparisons.
- str — Used for labels in the displayed output.

Input :
The program uses the following input values:
- Base rate = ₹30.00
- Fat percentage = 4.2%
- SNF percentage = 8.7%
- Milk quantity = 10 litres
- Standard fat = 3.5%
- Standard SNF = 8.5%

The values are assigned directly in the program.

Output :
The program displays the calculated fat steps, SNF steps, rate per litre and payment amount.



Evaluation :

The Python program was evaluated by substituting different values for milk quantity, fat percentage and SNF percentage into the mathematical expressions. The calculated fat steps and SNF steps were used to determine the rate per litre and the total payment.

For the given input values, the milk quantity is 10 litres, the fat percentage is 4.2%, and the SNF percentage is 8.7%.

Conclusion :

Arithmetic expressions and Python operators were used to calculate the fat steps, SNF steps, rate per litre and payment amount. The results demonstrate the importance of correct formula construction, operator precedence and rounding when implementing mathematical calculations in Python.

TASK 5

Aim :
To accept input from the user, convert input values into suitable data types and validate the entered milk quantity using Python input and conditional statements.

Problem Statement :
A milk payment calculator must accept information from the user instead of depending only on fixed values written in the program. The farmer ID is entered as text, while the milk quantity must be converted into a numerical value for further calculations.
Python's input() function returns the entered information as a string. Therefore, numerical input must be converted using functions such as int() or float(). The program must also check whether the quantity is valid before displaying the result.
The objective of this task is to demonstrate user input, type conversion, conditional statements and basic input validation.

Data Types :
- str — Stores the farmer ID and input text.
- float — Stores the quantity of milk.
- bool — Represents the result of the validation condition.
- int — Can store whole-number quantities when required.

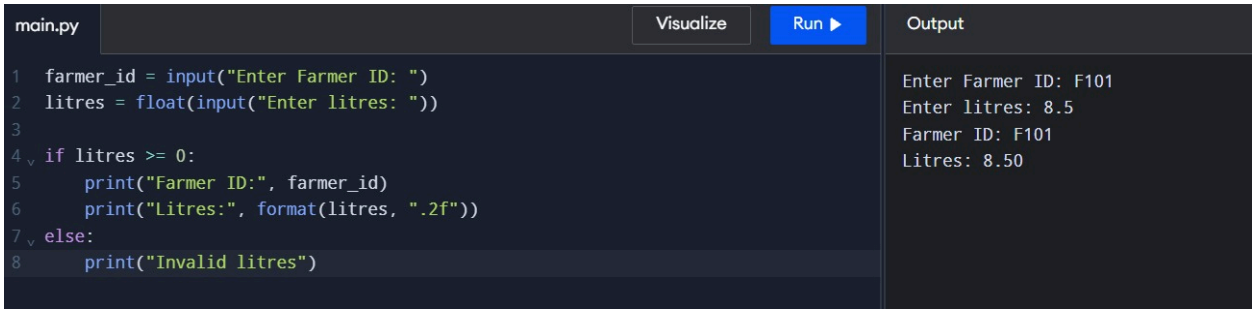
The input() function reads information from the user. The float() function converts a valid numerical string into a floating-point value. The if-else statement selects the appropriate action based on the condition.

Input :
The program accepts:
1. Farmer ID.
2. Milk quantity in litres.

The farmer ID is entered as a string, and the milk quantity is converted into a floating-point number.

Output :

If the milk quantity is zero or positive, the program displays the farmer ID and quantity with two decimal places. If the quantity is negative, the program displays an invalid-input message.



Evaluation :

The program checks the entered quantity using the condition litres >= 0. If the condition is true, the entered details are displayed. Otherwise, the error message is displayed.

The format(litres, ".2f") expression ensures that the quantity is displayed with two decimal places. For example, 8.5 is displayed as 8.50.

If the user enters text such as eight instead of a number, float() raises a ValueError. A more complete application can use try-except to handle such input errors and ask the user to enter a valid number.

Input validation is important because incorrect values can lead to incorrect payment calculations. Negative quantities must not be accepted, and the fat and SNF values should also be checked before the complete calculator is used.

Conclusion :

User input and type conversion were demonstrated successfully. Conditional statements were used to validate the milk quantity and display suitable output. The task shows how Python can accept user-provided data and perform basic validation before processing the information.

TASK 6

Aim :
To create and use user-defined functions in Python for calculating the rate per litre and the payment amount, improving the readability and reusability of the milk payment calculator.

Problem Statement :
A program that performs every calculation in one continuous block may become difficult to understand and maintain. When the same calculation is required for several farmers, repeating the same statements increases the length of the program and makes changes more difficult.
The objective of this task is to divide the milk payment calculator into separate functions. One function calculates the rate per litre, while another calculates the payment amount using the quantity and rate. This modular approach makes the program easier to test and modify.

Data Types :
- float — Stores milk quantity, fat percentage, SNF percentage, rate and payment.
- int — Stores rounded fat and SNF step counts.
- str — Stores labels and messages.
- tuple — Can hold multiple returned values when required.

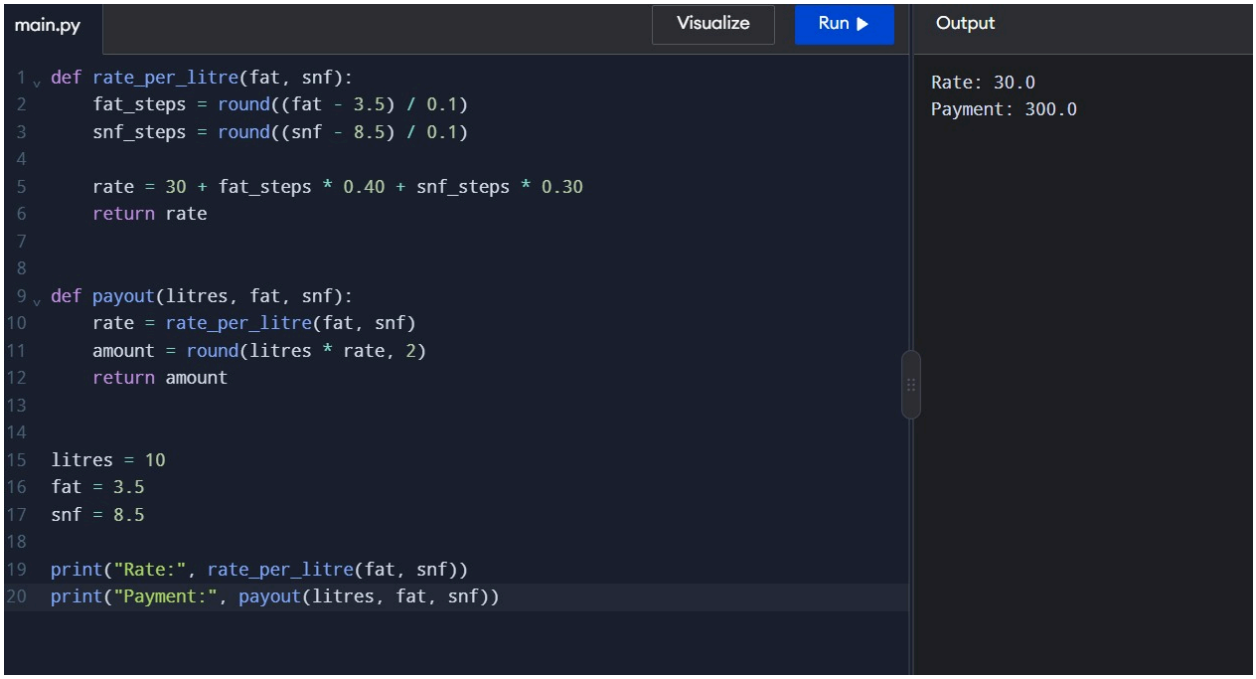
Functions used :
- rate_per_litre() calculates the milk rate.
- payout() calculates the payment amount.
- return sends a calculated value back to the calling statement.

Input :
The program uses the following values:
- Milk quantity = 10 litres
- Fat percentage = 3.5%
- SNF percentage = 8.5%

These values are passed as arguments to the functions.

Output :

The program displays the calculated rate per litre and the total payment for the supplied quantity.



Evaluation :

The rate_per_litre() function receives fat and SNF values, calculates their step counts and returns the resulting rate. The payout() function calls the rate calculation function, multiplies the rate by the quantity and returns the payment.

A function can be reused for multiple input combinations without rewriting the calculation statements. For example, payout(8.5, 4.2, 8.7) returns a payment of ₹283.90.

Functions also make debugging easier. If the rate calculation needs to be modified, the relevant function can be updated without changing the entire program.

For a single entry, each calculation takes a constant amount of time. For n entries, processing all entries takes $O(n)$ time.

Conclusion :

User-defined functions were created and used to calculate the milk rate and payment. The program demonstrates modular programming, function arguments, return values and code reuse. Dividing the program into functions improves readability, testing and maintainability.

TASK 7

Aim :
To create a reusable Python module containing milk rate and payment calculation functions and import the module into another Python program.

Problem Statement :
When a program contains several related functions, keeping all the code in a single file can make it difficult to maintain and reuse. A module allows related functions and variables to be stored in a separate Python file.
In the milk payment calculator, the rate and payment functions may be required by different programs. Instead of copying the same code into every program, the functions can be stored in a module and imported whenever required.
The objective of this task is to create a module, import it into another Python file and execute the functions provided by that module.

Data Types :
- float — Stores the milk rate and payment amount.
- int — Stores fat and SNF step counts.
- str — Stores module names and output labels.
- function — Represents reusable blocks of Python code.

Concepts used :
- Module creation
- import statement
- Function definition
- Module access using the dot operator
- Main guard

Input :
The module receives fat percentage, SNF percentage and milk quantity as function arguments. In the main program, the values used are:
- Fat = 4.2%
- SNF = 8.7%
- Milk quantity = 8.5 litres

Output :
The program displays the calculated rate per litre and payment amount by calling functions imported from the module.

main.py

Visualize | Run ▶

```python
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40 + snf_steps * 0.30
    return rate


def payout(litres, fat, snf):
    rate = rate_per_litre(fat, snf)
    return round(litres * rate, 2)


if __name__ == "__main__":
    print("Rate:", rate_per_litre(3.5, 8.5))
    print("Payment:", payout(10, 3.5, 8.5))
```

Output

```
Rate: 30.0
Payment: 300.0
```

main.py

Visualize | Run ▶

```python
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40 + snf_steps * 0.30
    return rate


def payout(litres, fat, snf):
    rate = rate_per_litre(fat, snf)
    return round(litres * rate, 2)


if __name__ == "__main__":
    print("Rate:", rate_per_litre(4.2, 8.7))
    print("Payment:", payout(8.5, 4.2, 8.7))
```

Output

```
Rate: 33.4
Payment: 283.9
```

main.py

Visualize | Run ▶

```python
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40 + snf_steps * 0.30
    return rate


def payout(litres, fat, snf):
    rate = rate_per_litre(fat, snf)
    return round(litres * rate, 2)


if __name__ == "__main__":
    print("Rate:", rate_per_litre(4.2, 8.7))
    print("Payment:", payout(12, 3.1, 8.3))
```

Output

```
Rate: 33.4
Payment: 333.6
```

Evaluation :

The statement import milkrate imports the module. The expression milkrate.rate_per_litre() calls the rate function, while milkrate.payout() calls the payment function.

The main guard, if __name__ == "__main__":, allows the sample statements inside the module to execute when the module is run directly. When imported by another file, those sample statements are not executed.

The two files must be saved in the same folder for this simple example to work without additional module-path configuration.

Modules improve code organisation and reduce duplication. If the rate formula changes, the function in the module can be modified without rewriting the programs that use it.

Conclusion :

A Python module was created and imported into another program successfully. The task demonstrates how modules and functions support code reuse, improve organisation and simplify the maintenance of the milk payment calculator.

TASK 8

Aim :
To design an algorithm, pseudocode and flowchart for the milk payment calculator before implementing the solution in Python.

Problem Statement :
A programming problem should be analysed before writing the actual code. An algorithm provides a clear sequence of steps, pseudocode represents those steps in a simple, language-independent format, and a flowchart represents the sequence using symbols and arrows.
For the milk payment calculator, the algorithm must read farmer information, validate the entered values, calculate fat and SNF steps, determine the rate, calculate the payment and display the total.
The objective of this task is to plan the calculation logically before implementing it in Python.

Data Types :
The proposed algorithm uses the following data types when implemented in Python:

- str — Farmer ID
- float — Milk quantity, fat percentage, SNF percentage and rate
- int — Rounded fat and SNF steps
- bool — Validation conditions
- list — Multiple farmer entries, if required

The variable total stores the running total of payments.

Input :
The algorithm receives the following information:

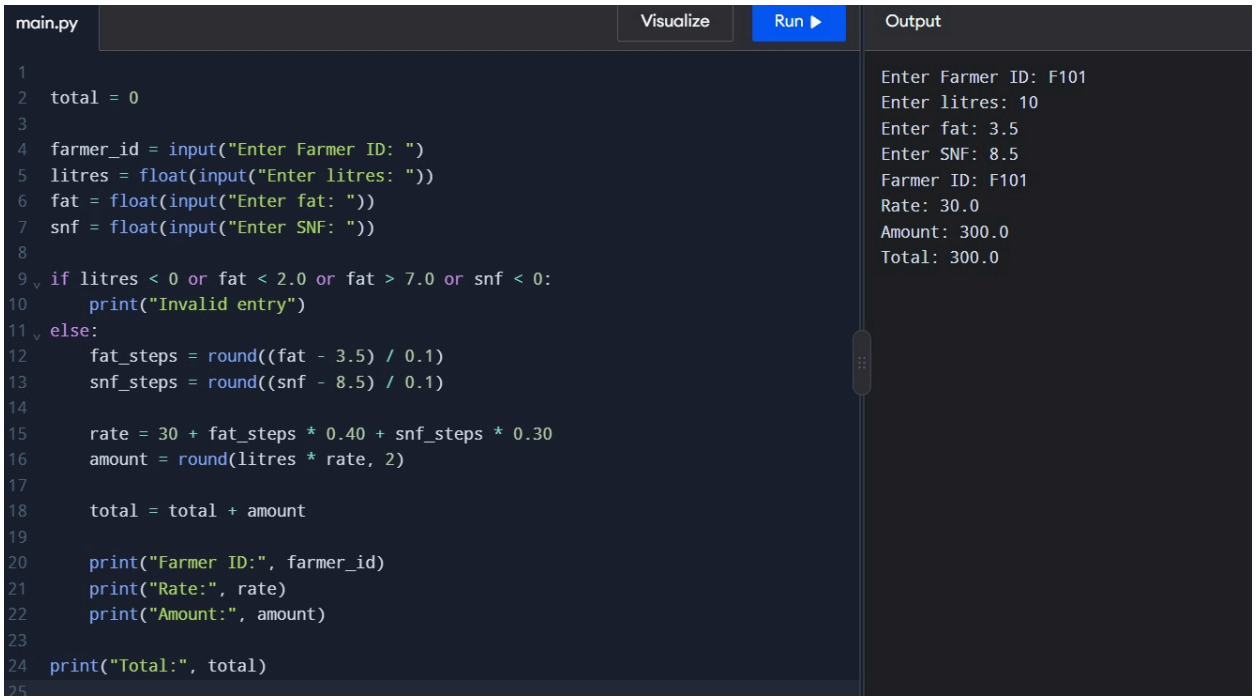
- Farmer ID
- Milk quantity in litres
- Fat percentage
- SNF percentage

The program checks whether the milk quantity is non-negative, the fat percentage is between 2.0% and 7.0%, and the SNF value is non-negative.

Output :

The algorithm produces:

- An error message if the input is invalid.
- The rate per litre if the input is valid.
- The payment amount.
- The running total of payments.



Algorithm :

1. Start the program.
2. Initialise total to zero.
3. Read the farmer ID, litres, fat and SNF.
4. Validate the input values.
5. If any value is invalid, display an error message.
6. Otherwise, calculate fat steps and SNF steps.
7. Calculate the rate per litre.
8. Calculate the payment amount.
9. Add the payment to the total.
10. Display the rate, amount and total.
11. Stop the program.

Evaluation of the algorithm :

For an input of 10 litres, 3.5% fat and 8.5% SNF, both step counts are zero. The rate is therefore ₹30.00 per litre, and the payment is ₹300.00.

The algorithm separates validation from calculation. This prevents invalid values from being used in the payment formula. The pseudocode is intended to describe the logic; it is not directly executable Python because words such as START, READ and DISPLAY are pseudocode instructions.

When converted into Python, these instructions are represented using statements such as input(), print() and if-else.

Conclusion :

The algorithm, pseudocode and flowchart were developed to represent the milk payment calculation process. The task demonstrates the importance of planning a solution before coding and provides a logical foundation for implementing and testing the Python program.

TASK 9

Aim :
To implement the milk payment calculator in Python and test the program using different valid, invalid and boundary input values.

Problem Statement :
A milk payment calculator must calculate the rate per litre and total payment correctly for different farmers. It must also reject invalid values to avoid incorrect calculations.
Testing is an important part of program development because it helps identify errors and verify whether the program meets the required conditions. Normal testing checks valid inputs, negative testing checks invalid inputs, and boundary testing checks values at the limits of the permitted range.
The objective of this task is to implement the calculation logic, validate the input values and evaluate the program using multiple test cases.

Data Types :
- float — Milk quantity, fat, SNF, rate and payment.
- int — Fat and SNF step counts.
- str — Error messages.
- bool — Results of validation conditions.
- tuple — Contains the returned rate and payment for valid inputs.

Input :
The program accepts three numerical inputs:
1. Milk quantity in litres.
2. Fat percentage.
3. SNF percentage.

The input values are passed to the calculate() function.

Output :
The program returns the calculated rate and payment for valid values. If an invalid value is entered, it returns an appropriate error message.

main.py

Visualize Run ▶

```python
def calculate(litres, fat, snf):
    if litres < 0:
        return "Invalid litres"

    if fat < 2.0 or fat > 7.0:
        return "Invalid fat percentage"

    if snf < 0:
        return "Invalid SNF"

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40 + snf_steps * 0.30
    amount = round(litres * rate, 2)

    return rate, amount


print(calculate(10, 3.5, 8.5))
print(calculate(8.5, 4.2, 8.7))
print(calculate(12, 3.1, 8.3))
print(calculate(-1, 3.5, 8.5))
print(calculate(10, 7.1, 8.5))
```

Output

```
(30.0, 300.0)
(33.4, 283.9)
(27.799999999999997, 333.6)
Invalid litres
Invalid fat percentage
```

Test case evaluation :

Test	Litres	Fats	SNF	Result
1	10	3.5	8.5	Rs.300.0
2	8.5	4.2	8.7	Rs.283.90
3	12	3.1	8.3	Rs.333.60
4	-1	3.5	8.5	Invalid litres
5	10	7.1	8.5	Invalid fat percentage
6	1	2.0	8.5	Rs.24.00
7	1	7.0	8.5	Rs.44.00

The test results show that valid entries produce the expected payments and invalid values are rejected. The minimum and maximum permitted fat values are accepted because the validation range is inclusive.

The program uses separate conditions for milk quantity, fat and SNF. This makes the error messages easier to understand and helps identify the reason for a rejected entry.

The program processes one entry at a time. If it is extended to process n entries, the overall processing time is $O(n)$, assuming each entry requires a fixed amount of calculation.

Conclusion :

The milk payment calculator was implemented and tested using valid, invalid and boundary values. The test cases demonstrated that the program calculates payments and detects invalid input conditions. Systematic testing improves the reliability and correctness of a Python application.

TASK 10

Aim :
To compare floating-point and decimal arithmetic and measure the execution time required to calculate the sum of multiple payment values.

Problem Statement :
A milk payment calculator may need to process hundreds or thousands of payment records. Although Python floating-point arithmetic is efficient, it may produce small representation errors when decimal fractions are added repeatedly.
Financial calculations require careful handling of decimal values. The Decimal class provides decimal arithmetic when values are created from strings. Execution time can be measured using Python's perf_counter() function.
The objective of this task is to compare the numerical results of both methods and measure their execution times using the same collection of payment values.

Data Types :
- list — Stores the payment values.
- str — Represents decimal amounts before conversion.
- float — Stores floating-point totals and measured times.
- Decimal — Stores decimal values.
- int — Represents the number of payment entries.

Input :
The program uses 1,000 payment values of ₹33.40 and three payment values of ₹0.10. The same values are used for both floating-point and decimal calculations to make the comparison meaningful.
The total expected amount is:
$1000 \times 33.40 + 3 \times 0.10$
$= 33400.00 + 0.30$
$= ₹33400.30$

Output :
The program displays the floating-point total, decimal total and execution time measured for each method.

main.py

Visualize Run ▶

```python
from decimal import Decimal
from time import perf_counter

values = ["33.40"] * 1000 + ["0.10"] * 3

start = perf_counter()
total1 = sum(float(x) for x in values)
time1 = perf_counter() - start

start = perf_counter()
total2 = sum(Decimal(x) for x in values)
time2 = perf_counter() - start

print("Float Total:", total1)
print("Decimal Total:", total2)
print("Float Time:", time1)
print("Decimal Time:", time2)
```

Output

```
Float Total: 33400.299999999996
Decimal Total: 33400.30
Float Time: 0.00016152999887708575
Decimal Time: 0.0003171699991071364
```

Evaluation :

The exact timing values depend on the computer, Python version and system load. Therefore, the execution times should be recorded after running the program on the actual system.

The floating-point total may be displayed as 33400.3 because that is a rounded representation of the result. Internally, the floating-point value may not be an exact decimal representation.

The decimal total is 33400.30, which represents the intended decimal amount exactly for this addition.

Both methods process every payment value, so their time complexity is $O(n)$, where n is the number of entries. The list uses $O(n)$ storage.

This experiment demonstrates the difference between numerical precision and execution speed. A method that is faster is not automatically the best choice if its numerical behaviour does not meet the application's requirements.

Conclusion :

Floating-point and decimal arithmetic were compared using a collection of payment values. The experiment demonstrated the importance of numerical precision in financial calculations and provided a method for measuring execution time. Decimal arithmetic is preferable when exact decimal monetary calculations are required, while actual execution times must be measured on the target computer.

TASK 11

Aim :
To evaluate the existing milk payment calculator and recommend a suitable Python design based on accuracy, maintainability, validation, reusability and computational efficiency.

Problem Statement :
A milk collection centre may receive milk from many farmers every day. The payment calculator must process the quantity of milk, fat percentage and SNF percentage accurately. It should also be easy to maintain when the rate card changes or additional requirements are introduced.
A poorly organised program may contain repeated calculations, inconsistent rate values and insufficient input validation. Such problems can make the program difficult to update and test.
The objective of this task is to evaluate different design choices and recommend a modular approach that can support reliable milk payment calculations. The design should separate the calculation logic from input handling and record management.

Data Types :
The recommended design uses the following data types and structures.

Data Type	Application
str	Farmer ID and messages
float	Measurement values in a basic prototype
decimal	Monetary calculations
int	Step counts and record counts
list	Stores multiple farmer records
dict	Stores named fields or rate-card information
bool	Represents validation results

For a financial application, Decimal should be constructed from strings or other exact decimal values rather than from approximate floating-point values.

Input :
The recommended design accepts:

- Farmer ID
- Quantity of milk
- Fat percentage
- SNF percentage
- Applicable rate card
- Date or period of collection, when required

For a simple prototype, the rate card can be defined using named constants. A larger application may load the rate card from a validated configuration file or database.

Output :
The program should produce:

- Validated farmer details
- Fat and SNF step counts
- Rate per litre
- Payment amount
- Total payment for the selected period
- Error messages for invalid entries

For an actual milk collection system, the output should also support record verification and payment auditing.

Evaluation :
Recommended design

The following principles should be applied when developing the final application.
1. Use named constants.
The base rate, standard fat and standard SNF values should be stored in one place. This avoids repeating the same numerical values throughout the program.
2. Use separate functions.
Functions should be created for input validation, rate calculation, payment calculation and total calculation. This improves readability and allows each function to be tested independently.
3. Use appropriate numerical types.
Measurement values may be processed as decimal measurements, while monetary values should use Decimal or integer paise to avoid unwanted floating-point rounding errors.
4. Validate all input values.
The program should reject negative milk quantities, fat values outside the specified range and invalid SNF values. The official SNF limits should be confirmed before deployment.
5. Store records systematically.

A list or database can store the farmer ID, quantity, readings, rate and payment. This supports future reporting, corrections and verification.

6. Test every modification.

Whenever a rate or calculation rule changes, the existing test cases should be executed again. This helps ensure that new modifications do not affect previously correct behaviour.

Conclusion :

A modular and maintainable design was recommended for the milk payment calculator. Named constants, reusable functions, suitable numerical types, input validation and systematic record storage improve the reliability of the application. The evaluation also demonstrated how time complexity and storage requirements can be considered when extending the program to process many farmer records.

TASK 12

Aim :

To modify the existing milk payment calculator to support seasonal rate changes, two-decimal measurement readings and a bonus for farmers whose total milk supply exceeds 500 litres over ten days.

Problem Statement :

The original milk payment calculator uses a base rate of ₹30 per litre, a fat step value of ₹0.40 per 0.1% and an SNF step value of ₹0.30 per 0.1%.

The revised requirements introduce seasonal pricing and a bonus system. During April to June, the base rate changes to ₹32 per litre and the fat step value changes to ₹0.45. The SNF step value remains ₹0.30 because no change has been specified.

The system must also accept readings containing two decimal places. Farmers supplying more than 500 litres over ten days receive a bonus of ₹0.50 per litre.

The objective is to adapt the original design without unnecessarily repeating the rate and payment calculation logic.

Data Types :

- float — Stores measurements and intermediate calculations in the basic implementation.
- int — Stores rounded fat and SNF step counts.
- str — Stores the selected season.
- list — Stores the ten-day collection records.
- tuple — Stores individual records containing litres, fat and SNF.
- bool — Determines whether the total quantity qualifies for a bonus.

For a financial implementation, Decimal is preferable for monetary calculations and final rounding.

Input :

The program accepts a collection of ten-day records. Each record contains:

1. Milk quantity in litres.
2. Fat percentage.
3. SNF percentage.

It also accepts the season, which determines the rate card to be used.

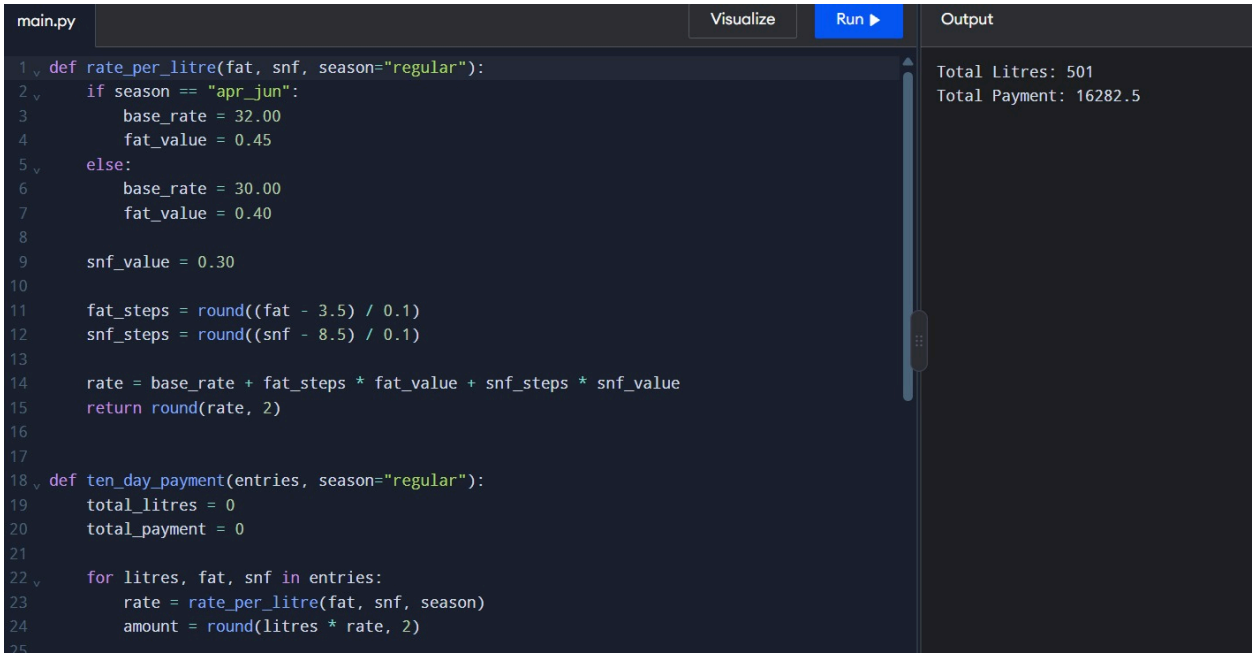
For this example, the records are:

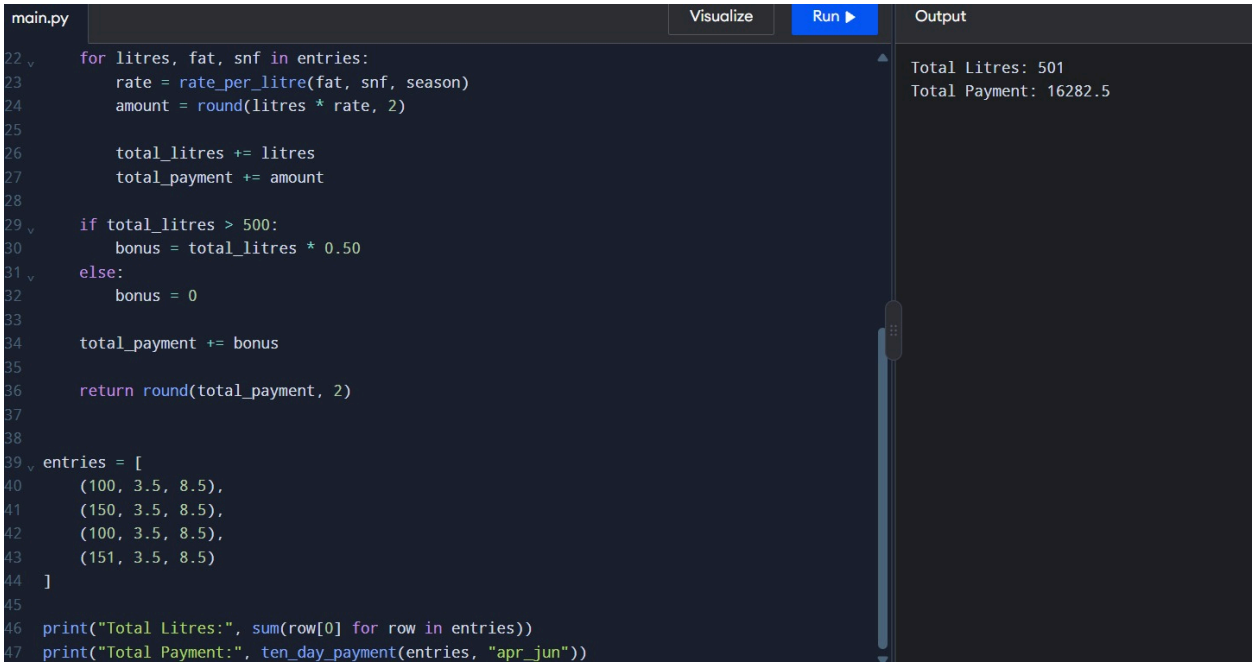
Entry	Litres	Fat	SNF
1	100	3.5	8.5
2	150	3.5	8.5
3	100	3.5	8.5
4	151	3.5	8.5
Total	501	-	-

The total quantity exceeds 500 litres, so the bonus condition is satisfied.

Output :

The program displays the total quantity of milk supplied over ten days and the total payment after applying the relevant seasonal rate and bonus.





Evaluation :

Formulae

For the April–June season: $R = 32 + (F_s \times 0.45) + (S_s \times 0.30)$

For the regular season: $R = 30 + (F_s \times 0.40) + (S_s \times 0.30)$

The program selects the rate card based on the season. It then calculates each payment, adds the quantities and checks whether the total qualifies for a bonus.

The bonus is applied to all litres once the threshold is exceeded. This is an assumption based on the stated requirement; the actual policy should be confirmed with the faculty or co-operative.

The program uses separate functions for seasonal rate calculation and ten-day payment calculation. This structure allows the rate card or bonus rule to be modified independently.

For financial deployment, the program should be upgraded to use Decimal for monetary values and explicit decimal rounding. It should also validate each record before calculating payments.

Conclusion :

The milk payment calculator was adapted to include seasonal base rates, a revised fat step value and a ten-day bonus system. The example demonstrated how the revised rules affect the rate and final payment. Separating seasonal pricing and bonus calculation into functions makes the program easier to modify, test and maintain.